

Q1

Summary

The paper *Predicting firm performance and size using machine learning with a Bayesian perspective* (Science Direct: S2666827023000063) uses supervised learning to predict firm sales and firm profits from financial inputs for registered food manufacturing firms in India. It compares four tree-based regression methods: a single regression tree, a bootstrap forest, boosted trees, and BART (Bayesian Additive Regression Trees). They evaluate the predictive strength by comparing the R^2 of each model across ten rounds of validation. The study found BART to be the best predictor.

Furthermore, the study found sales easier to predict than profits with more consistent results. The greater unpredictability of profits likely reflects additional firm-level decisions such as marketing strategies or management choices that are not captured in the dataset, leading to noisier results.

Research questions

The paper asks:

1. Can modern machine learning models predict standard firm sales (top line) and profits (bottom line) for registered manufacturing firms in a developing country?
2. Are sales more predictable than profits for these firms?

Methods

Every model in the paper is a kind of supervised learning method for regression as we learned: y is regressed on x with the goal to learn a prediction function that approximates the relationship between firm characteristics and firm sales/profits.

The simplest model we studied was linear regression, which assumes a linear relationship between predictors and the responses. The methods used in this paper instead rely on tree-based machine learning regression models, which allow for more flexible non-linear relationships. So all four approaches estimate the same predictive function, but differ in their flexibility and how they control overfitting.

Regression tree: A tree recursively splits the sample space into regions that minimise prediction error then assigns (predicts) a mean outcome in each region. However, single trees tend to have high variance (overfitting) due to sensitivity when splitting regions which perpetuates with the size of the tree.

Course link: a more developed regression than linear regression; uses non-linear relationships instead of manually adding squared interactions or using local regression.

Bootstrap forest / bagging: Bagging (bootstrap aggregating) averages many regression trees on data samples bootstrapped from the original data, then averages them. The method of averaging across the "forest" of trees reduces the risk of overfitting seen with single regression trees.

Course link: As was demonstrated in multiple contexts e.g. in LOWESS weighting and the SVM cost parameter, it's important to consider flexible vs simple models, regularisation, overfitting and how to minimise it.

Boosted trees: Another ensemble tree model which sequentially builds multiple small trees wherein each is trained on the residual errors of the previous tree.

Course link: this connects to the theme of reducing overfitting, which we discussed in the context of SVMs and neural networks. It's reminiscent of the weight decay in neural networks and the β^2 term in SVM.

BART: BART (Bayesian Additive Regression Trees) is similarly a "forest" but in this case, each tree and its parameters have a prior weighting assigned according to Bayesian probability calculations as a means of regularisation.

Course link: it's like Naive Bayes wherein Bayes theorem is used to assign probabilities to the likeliness of an outcome, but here, it's applied to regression trees.

Experiment and Results

The data was from CME and provided by the Centre for Monitoring Indian Economy for fiscal years 2006-07 to 2018-19, focusing on registered food manufacturing.

The dependent variable was sales deflated by the wholesale price index of food products and profits before depreciation, interest, tax and amortisation deflated by wholesale price index of food products. The independent variables for the analysis were grouped under four categories where the classification was based on the different measures of firm size. The dataset consisted of 337 data points with 14 independent variables.

Execution:

1. Data preparation
 - Pull firm accounts data
 - Deflate sales
2. Model Training
 - Make four predictor categories
 - Fit model (tree / forest / boosted / BART)
3. Evaluation
 - Run ten validation studies
 - Find R^2
 - Identify the most important predictors
 - Compare sales vs profits predictability

R-squared Results:

Model	Sales	Profit
Regression Tree	0.100 - 0.948	0.004 - 0.754
Bootstrap Forest	0.616 - 0.969	0.125 - 0.746
Boosted Trees	0.748 - 0.976	0.019 - 0.750
BART	0.762 - 0.997	0.300 - 0.970

BART provided the most accurate prediction overall, though profit proved much more difficult to predict than sales.

Evaluation

What Went Well:

- Clear applied question and relevant use of machine learning to study firm performance.
- Sensible comparisons within the same family of tree-based models.
- Use of validation to facilitate accuracy of R scores.

Even Better If:

- More explanation was given about how the models were tuned. The paper compares several models but does not clearly describe how parameters such as tree depth or number of trees were chosen.
- No detailed description of the validation process. This makes it unclear whether the high R^2 values for sales reflect true predictive performance or just easy training-testing splits.
- The paper compares several complex models but does not include a simple baseline such as linear regression (OLS). Including a simpler model would make it easier to judge how much improvement the machine learning methods provide.

Practical extensions:

- Add an OLS baseline (with ridge/lasso regularisation to handle correlated variables).
- Add a clear warning regarding importance $\neq$ causation in accounting data.
- Repeat the study on a different data set e.g. from a different country, time period etc and see if the results repeat.

Conclusion

The paper provides applied evidence that flexible nonlinear machine learning models can predict firm sales accurately using accounting data, and that profits are more nuanced and difficult. It would be stronger with slightly more detailed experimental reporting and simple baselines for reference.

References

- Saha, Young & Thacker (2023). Predicting firm performance and size using machine learning with a Bayesian perspective. *Machine Learning with Applications*, 11, 100453.

- Chipman, George & McCulloch (2010). BART: Bayesian additive regression trees. *Annals of Applied Statistics*, 4(1), 266–298.
- Friedman (2001). Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Breiman (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140.

Q2

Summary

This analysis uses supervised learning methods to predict the probability of default among Taiwanese credit card users.

Data

The Taiwan credit default dataset has 30,000 clients and 23 predictors. The response variable is `default.payment.next.month` (1 on default, 0 otherwise).

Predictors include:

- demographics (SEX, EDUCATION, MARRIAGE, AGE)
- credit limit (LIMIT_BAL)
- six months of past repayment status (PAY_0 to PAY_5)
- six months of bill statements (BILL_AMT1 to BILL_AMT6)
- six months of past payments (PAY_AMT1 to PAY_AMT6)

Note that features like age (years) and bill amounts (in NT dollars) are on very different scales. Approximately 22% of clients defaulted.

Models

Preprocessing

First, the ID column and header row were manually removed from the dataset. Then, it was split 2:1 training to testing.

Model 1: Logistic Regression (GLM)

Generalised Linear Models (GLMs) extend ordinary linear regression to accommodate non-normal data using maximum likelihood estimation. One such case suited to a GLM is when the responses follow a Bernoulli distribution (i.e. binary outcome).

$$y \sim \text{Ber}(\pi)$$

In this case, the GLM is called a logistic regression which relates the predictors to the response variables via the logit link function

$$\text{logit}(\pi) = \log(\pi / (1 - \pi))$$

where $\pi(x) = P(Y=1|X)$.

Algorithms based on modeling such conditional probabilities are known as discriminative learning algorithms.

Model 2: Support Vector Machines (SVMs)

Support Vector Classifiers attempt to find a linear boundary that separates two classes while maximising the margin between them. The classification is based on the linear function

$$f(x) = \beta_0 + \beta^T x$$

SVMs extend this idea by enlarging the feature space using kernel functions. It projects the data into a higher-dimensional feature space and computes a separating hyperplane that can capture nonlinear relationships between the predictors and the outcome.

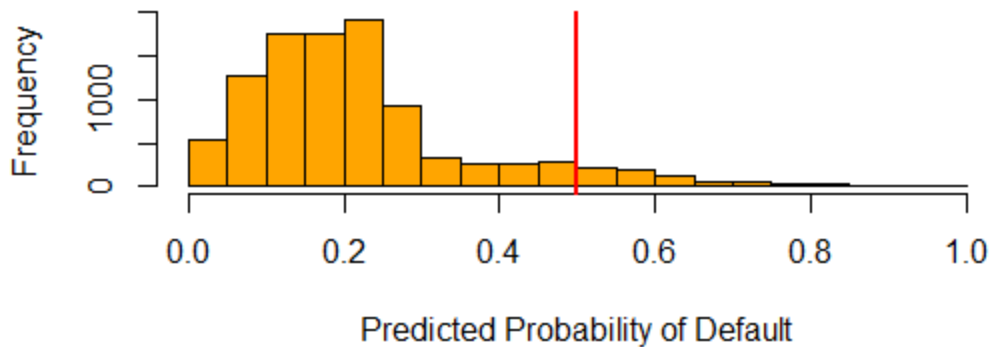
In this analysis a linear kernel was used, meaning the decision boundary remains linear, followed by a (Gaussian) Radial Basis Function (RBF) which allows for a non-linear decision boundary.

Results

Logistic Regression

Logistic regression achieved about 81% accuracy on the test data. The model demonstrated higher accuracy when identifying non-defaults (97.5%) compared to defaults (23.0%). Considering the data is dominated by non-defaults, this outcome is unsurprising as it means the model had considerably less default data with which to learn. The model required 0.35s to run.

LR Predicted Default Probabilities



Predicted	Actual 0	Actual 1
0	7595	1702
1	195	508

SVM

Tuning

A range of cost hyperparameters were tested, from which the value 0.1 was chosen. The tuning was set to 2-fold cross-validation (instead of the default 10-fold) for the sake of speed and to avoid a memory error. Scaling was set to True, as the predictors within the set have very different scales and this can negatively affect the models, especially RBF.

cost	error	dispersion
0.01	0.1916	0.0050911688
0.10	0.1915	0.0055154329
1	0.1918	0.0039597980
10	0.1939	0.0001414214

Linear Kernel

SVM with a linear kernel gave a similar overall accuracy of 80%. Again, the model demonstrated higher accuracy when identifying non-defaults (97.3%) than defaults (22.4%) reflecting the imbalance of non-default vs default data within the dataset. Considering the model is set to a linear kernel, the decision boundary is a linear hyperplane. Logistic regression also produces a linear boundary, so it is unsurprising that the results are very similar. It took 68s to run.

Predicted	Actual 0	Actual 1
0	7581	1714
1	209	496

RBF Kernel

Lastly, SVM was run with an RBF kernel to test if there's a detectable non-linear relationship between the predictors and regressors. Overall accuracy was again around 82%. The gamma hyperparameter was selected using a common rule of thumb: one divided by the number of predictors. This dataset has 23 predictors which means $\gamma \sim 0.043$.

Results strongly suggest a non-linear structure within the data as non-defaults were predicted with 95.9% accuracy while defaults were predicted with 32.0% accuracy. This means RBF remained highly accurate when detecting non-defaults but substantially improved the detection of defaults compared to both logistic regression and a linear kernel. This model took 120s to run.

Predicted	Actual 0	Actual 1
0	7467	1503
1	323	707

Conclusion

In conclusion, the linear models performed similarly, but there was a notable increase in the accuracy of the RBF model which indicates some non-linear structure within the data.

Logistic regression has the advantage of simplicity, speed, and it provides explicit probability estimates, which is useful for interpretability. However, the extra configurability and flexibility of SVM can outperform linear models with data that exhibit non-linear structure, as was the case here.

These results suggest that while default behaviour is largely predictable with linear modelling, some nonlinear patterns may also exist.

```
## Appendix ##
```

```
library(e1071)
```

```
# https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients
```

```
data = read.csv("default of credit card clients.csv")
```

```
data$default.payment.next.month = as.factor(data$default.payment.next.month)
```

```
set.seed(1) # reproducible
```

```
n = nrow(data) # count how many rows of data
```

```
pivot = sample(1:n, size = (2/3)*n) # find the index separating 2:1
```

```
train = data[pivot,] # 20000 training data points
```

```
test = data[-pivot,] # 10000 to make predictions on
```

```
n_test = nrow(test)
```

```
# 1) Logistic Regression
```

```
system.time({lr_train = glm(default.payment.next.month ~ .,
```

```
data = train,
```

```
family = "binomial")
```

```
lr_pred = predict(lr_train, newdata = test, type = "response") # predict>50% = 1, 0 otherwise
```

```

})
res = table(Predicted = ifelse(lr_pred > 0.5, 1, 0), Actual = test$default.payment.next.month)
FF = res[1,1]
FT = res[1,2]
TF = res[2,1]
TT = res[2,2]

print('Non-default prediction accuracy:')
FF / (FF + TF)
print('Default prediction accuracy:')
TT / (TT + FT)
print('Overall accuracy:')
(TT + FF) / n_test

# histogram of predicted probabilities
hist(lr_pred,
      breaks = 30,
      main = "LR Predicted Default Probabilities",
      xlab = "Predicted Probability of Default",
      col='orange'
    )
abline(v = 0.5, col = "red", lwd = 2) # default vs non-default threshold

# 2) SVM
# hyperparameter tuning (linear)
train_cut = train[1:(5000*2),] # chop to save time since we're running 8 times (4 costs x 2
folds)
pick_c = tune(svm,
              default.payment.next.month ~ .,
              data = train_cut,
              kernel = "linear",
              scale = T,
              ranges = list(cost = c(0.01, 0.1, 1, 10)),
              tunecontrol = tune.control(cross = 2)
            )
pick_c$performances # see how each cost param performed

# Linear kernel
system.time({
  svm_train = svm(default.payment.next.month ~ .,
                  data = train,
                  kernel = "linear",
                  cost = 0.1,
                  scale = T)
  svm_pred = predict(svm_train, test)
})
res=table(Predicted = svm_pred, Actual = test$default.payment.next.month)

```



```

FF = res[1,1]
FT = res[1,2]
TF = res[2,1]
TT = res[2,2]

print('Non-default prediction accuracy:')
FF / (FF + TF)
print('Default prediction accuracy:')
TT / (TT + FT)
print('Overall accuracy:')
(TT + FF) / n_test

# rbf kernel
system.time({
  svm_train = svm(default.payment.next.month ~ .,
                  data = train,
                  kernel = "radial",
                  cost = 0.1,
                  scale = T,
                  gamma=(1/23)) # gamma rule of thumb: 1/no. of predictors
  svm_pred = predict(svm_train, test)
})
res=table(Predicted = svm_pred, Actual = test$default.payment.next.month)
FF = res[1,1]
FT = res[1,2]
TF = res[2,1]
TT = res[2,2]

print('Non-default prediction accuracy:')
FF / (FF + TF)
print('Default prediction accuracy:')
TT / (TT + FT)
print('Overall accuracy:')
(TT + FF) / n_test

```

References

- Matuozzo, A. (2025). Statistical Learning. Lecture notes, Birkbeck, University of London.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An Introduction to Statistical Learning: With Applications in R. Springer, New York.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.